Parallel Processing Lab

MPI Solutions with Code-by-Code Explanations

18 lab problems • Question statements • C/MPI code • Explanation • Important limitations

How to use this document	Each problem includes a cleaned question based on the uploaded lab list, the supplied C/MPI solution, a step-by-step explanation, and notes about assumptions or limitations.
Compile	Save one program at a time as a .c file and compile with: mpicc program.c -o program
Run	Example for four processes: mpirun -np 4 ./program. Some MPI installations use mpiexec instead of mpirun.
Important	The code is based on the provided solution text. Where a solution does not fully meet the lab requirement or has a correctness limitation, this guide flags it instead of silently presenting it as complete.

Contents

1. Boundary Exchange with Point-to-Point Communication
2. Matrix Row Distribution and Row Sums
3. Ring Topology Communication
4. Ping-Pong: Latency and Round-Trip Time
5. Distributed Matrix Multiplication
6. Halo Exchange and Stencil Computation
7. Nonblocking Vector Exchange
8. Avoiding Deadlock
9. Distributed Sorting
10. Producer–Consumer with MPI

11. Broadcast Configuration Parameters
12. Scatter, Compute, and Gather
13. Uneven Distribution with Scatterv and Gatherv
14. Global Sum, Maximum, Minimum, and Average
15. Trapezoidal Rule Integration
16. Prefix Sum with MPI_Scan
17. All-to-All Communication
18. Global Histogram with MPI_Allreduce

Problem 1: Boundary Exchange with Point-to-Point Communication

Question

Divide an integer array among MPI processes. Each process exchanges boundary elements with neighboring processes using MPI_Send and MPI_Recv, then reports the exchanged values.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int local_data[3] = {rank*30 + 10, rank*30 + 20, rank*30 + 30};
    int left_recv = -1, right_recv = -1;

    if (rank % 2 == 0) {
        if (rank < size - 1) MPI_Send(&local_data[2], 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD);
        if (rank > 0) MPI_Recv(&left_recv, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        if (rank > 0) MPI_Send(&local_data[0], 1, MPI_INT, rank - 1, 1, MPI_COMM_WORLD);
        if (rank < size - 1) MPI_Recv(&right_recv, 1, MPI_INT, rank + 1, 1, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
    } else {
        if (rank > 0) MPI_Recv(&left_recv, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        if (rank < size - 1) MPI_Send(&local_data[2], 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD);
        if (rank < size - 1) MPI_Recv(&right_recv, 1, MPI_INT, rank + 1, 1, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        if (rank > 0) MPI_Send(&local_data[0], 1, MPI_INT, rank - 1, 1, MPI_COMM_WORLD);
    }

    if (rank < size - 1) printf("P%d: Right boundary sent = %d\n", rank, local_data[2]);
    if (rank > 0) printf("P%d: Left boundary received = %d\n", rank, left_recv);
    if (rank == size - 1) printf("Boundary exchange completed successfully.\n");

    MPI_Finalize();
    return 0;
}
```

Code Explanation

MPI setup. MPI_Init starts MPI. MPI_Comm_rank gives each process its rank, and MPI_Comm_size gives the total number of processes.

Local data. Each process creates three integers based on its rank. For example, rank 0 gets 10, 20, 30; rank 1 gets 40, 50, 60.

Boundary exchange. The last local element is sent to the right neighbor and the first local element is sent to the left neighbor. The parity-based ordering attempts to coordinate blocking sends and receives.

Output and shutdown. The program prints sent and received values, then MPI_Finalize closes MPI.

Check before submission: Check the communication order carefully for your MPI implementation and process count. The supplied code only prints the left receive and does not print right_recv; it also uses a parity-based blocking schedule that is harder to maintain than MPI_Sendrecv.

Problem 2: Matrix Row Distribution and Row Sums

Question

The root process distributes one matrix row to each process using MPI_Send. Each process computes its row sum and sends the sum back to the root, which calculates the total matrix sum.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int matrix[4][4] = {{1,2,3,4}, {5,6,7,8}, {9,10,11,12}, {13,14,15,16}};
    int local_row[4];

    if (rank == 0) {
        for (int i=0; i<4; i++) local_row[i] = matrix[0][i];
        for (int i=1; i<size; i++) MPI_Send(matrix[i], 4, MPI_INT, i, 0, MPI_COMM_WORLD);
    } else {
        MPI_Recv(local_row, 4, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }

    int sum = 0;
    for (int i=0; i<4; i++) sum += local_row[i];
    printf("P%d: Row = %d %d %d %d, Sum = %d\n", rank, local_row[0], local_row[1], local_row[2],
local_row[3], sum);

    if (rank == 0) {
        int total = sum, recv_sum;
        for (int i=1; i<size; i++) {
            MPI_Recv(&recv_sum, 1, MPI_INT, i, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            total += recv_sum;
        }
        printf("Total matrix sum = %d\n", total);
    } else {
        MPI_Send(&sum, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Matrix and row storage. The root stores a 4 × 4 matrix. local_row is the four-element buffer used by each process.

Distribute rows. Rank 0 copies row 0 into its local buffer and sends rows 1–3 to ranks 1–3. Workers receive one row with MPI_Recv.

Calculate row sums. Every process loops over its four local values and adds them into sum.

Return and combine. Workers send their row sums to rank 0. The root adds its own sum and all received sums to produce the total.

Check before submission: The code assumes exactly four processes. Run with -np 4; fewer processes leave rows unprocessed, and more processes can access beyond matrix[4][4]. The root's local row is copied correctly, but the solution does not broadcast or otherwise validate the expected process count.

Problem 3: Ring Topology Communication

Question

Create a ring in which each process receives a message from its previous process and forwards it to the next process. Process 0 starts with value 100, and the message returns to process 0.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int msg = (rank == 0) ? 100 : 0;
    int next = (rank + 1) % size;
    int prev = (rank - 1 + size) % size;

    if (rank == 0) {
        MPI_Send(&msg, 1, MPI_INT, next, 0, MPI_COMM_WORLD);
        printf("P0 sends 100 to P1\n");
        MPI_Recv(&msg, 1, MPI_INT, prev, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("P0 receives 100 from P3\nRing communication completed.\n");
    } else {
        MPI_Recv(&msg, 1, MPI_INT, prev, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("P%d receives 100 from P%d\n", rank, prev);
        MPI_Send(&msg, 1, MPI_INT, next, 0, MPI_COMM_WORLD);
        printf("P%d sends 100 to P%d\n", rank, next);
    }

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Find neighbors. next = (rank + 1) % size wraps the last process around to rank 0. prev = (rank - 1 + size) % size wraps rank 0 around to the last process.

Start the token. Only rank 0 initializes msg to 100; other processes initialize it to 0 until they receive the message.

Forward around the ring. Rank 0 sends to next and then receives from prev. Other ranks receive from prev and forward the received value to next.

Finish. The message should travel P0 → P1 → … → P(size−1) → P0.

Check before submission: The message value is forwarded unchanged, but the hard-coded final message text says 'P3', so it is only correct for four processes. The exercise asks to measure communication time for different message sizes; this code does not perform timing or vary message size.

Problem 4: Ping-Pong: Latency and Round-Trip Time

Question

Implement repeated message exchanges between two processes. Measure the average round-trip time and estimate the one-way time for a 1,024-byte message over five iterations.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int iterations = 5;
    int msg_size = 1024;
    char buffer[1024] = {0};

    double start_time, end_time;

    if (rank == 0) start_time = MPI_Wtime();

    for (int i = 0; i < iterations; i++) {
        if (rank == 0) {
            MPI_Send(buffer, msg_size, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
            MPI_Recv(buffer, msg_size, MPI_CHAR, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        } else if (rank == 1) {
            MPI_Recv(buffer, msg_size, MPI_CHAR, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            MPI_Send(buffer, msg_size, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
        }
    }

    if (rank == 0) {
        end_time = MPI_Wtime();
        double total_time = end_time - start_time;
        printf("Message size : %d bytes\nIterations : %d\n", msg_size, iterations);
        printf("Average round-trip : %f seconds\n", total_time / iterations);
        printf("Average one-way : %f seconds\n", (total_time / iterations) / 2.0);
    }

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Message buffer. buffer is a character array holding the message bytes. msg_size determines how many bytes MPI transfers; iterations controls the number of round trips.

Timer. MPI_Wtime returns a wall-clock timer. Rank 0 records the start time before the loop and the end time after all exchanges.

Ping-pong loop. Rank 0 sends to rank 1 and waits for the reply. Rank 1 receives and sends the message back.

Metrics. Total elapsed time divided by iterations gives average round-trip time. Dividing that value by two estimates average one-way time; this estimate is not a separate one-way measurement.

Check before submission: The buffer has capacity 1,024 bytes, so msg_size must not exceed 1,024. The current program only tests one size and does not calculate bandwidth. Increase the buffer dynamically or set a safe maximum to test megabyte messages.

Lab analysis note: For bandwidth, a common estimate is message_size_bytes / average_one_way_seconds. This is only meaningful when the time is large enough to measure reliably; use many iterations and repeat each size.

Problem 5: Distributed Matrix Multiplication

Question

Distribute rows of matrix A among MPI processes, calculate the corresponding rows of C = A × B, and return the results to the root.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int A[2][2] = {{1, 2}, {3, 4}};
    int B[2][2] = {{5, 6}, {7, 8}};
    int C[2][2];
    int local_A[2], local_C[2] = {0, 0};

    if (rank == 0) {
        for (int i=0; i<2; i++) local_A[i] = A[0][i];
        MPI_Send(A[1], 2, MPI_INT, 1, 0, MPI_COMM_WORLD);
    } else {
        MPI_Recv(local_A, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            local_C[i] += local_A[j] * B[j][i];
        }
    }

    if (rank == 0) {
        for (int i=0; i<2; i++) C[0][i] = local_C[i];
        MPI_Recv(C[1], 2, MPI_INT, 1, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Matrix C:\n%d %d\n%d %d\n", C[0][0], C[0][1], C[1][0], C[1][1]);
    } else {
        MPI_Send(local_C, 2, MPI_INT, 0, 1, MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Matrices and local buffers. A and B are 2 × 2 matrices. Each process receives one row of A and calculates one row of C.

Distribute A. Rank 0 uses its first row locally and sends the second row to rank 1.

Multiply a row. The nested loops calculate dot products between the local row of A and columns of B.

Collect results. Rank 0 stores its computed row, receives rank 1's row, and prints C.

Check before submission: The multiplication loop uses B[j][i], which computes row-vector times B transposed relative to the usual C[i][j] = sum_k A[i][k] * B[k][j]. For the supplied symmetric-looking example the expected result should be checked carefully; the usual correct indexing is local_C[j] += local_A[k] * B[k][j].

Problem 6: Halo Exchange and Stencil Computation

Question

Split a one-dimensional array across processes. Exchange halo values with adjacent processes and calculate a local stencil using neighboring values.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int local[2] = {(rank * 20) + 10, (rank * 20) + 20};
    int left_halo = 0, right_halo = 0;

    if (rank > 0) MPI_Send(&local[0], 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD);
    if (rank < size - 1) MPI_Recv(&right_halo, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

    if (rank < size - 1) MPI_Send(&local[1], 1, MPI_INT, rank + 1, 1, MPI_COMM_WORLD);
    if (rank > 0) MPI_Recv(&left_halo, 1, MPI_INT, rank - 1, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    int out[2];
    out[0] = left_halo + local[0] + local[1];
    out[1] = local[0] + local[1] + right_halo;

    printf("P%d Stencil result: %d %d\n", rank, out[0], out[1]);

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Local segment. Each process holds two adjacent values. left_halo and right_halo are intended to store neighboring values outside that segment.

Exchange halos. Processes send boundary values to neighbors using point-to-point communication. Ranks at the ends skip communication with nonexistent neighbors.

Stencil. Each output uses a local value and its neighbor(s). The exact boundary convention must be defined carefully: the supplied code uses local[0] + local[1] for the first output and local[0] + local[1] + right_halo for the second.

Print. Each process prints its two local output values.

Check before submission: The supplied send/receive order can deadlock for some process counts because neighboring ranks may both block in sends. Also, the stated stencil out[i] = left + current + right is not implemented as a three-point stencil for each global element. Use MPI_Sendrecv or nonblocking exchange and explicitly define endpoint boundary values for a robust version.

Problem 7: Nonblocking Vector Exchange

Question

Exchange vectors between two processes with MPI_Isend and MPI_Irecv, wait for both operations to complete, and compare the timing with blocking communication.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int send_arr[8], recv_arr[8];
    for (int i = 0; i < 8; i++) send_arr[i] = (rank == 0) ? (i+1)*10 : (i+1);

    MPI_Request reqs[2];
    int target = (rank == 0) ? 1 : 0;

    double start = MPI_Wtime();
    MPI_Isend(send_arr, 8, MPI_INT, target, 0, MPI_COMM_WORLD, &reqs[0]);
    MPI_Irecv(recv_arr, 8, MPI_INT, target, 0, MPI_COMM_WORLD, &reqs[1]);
    MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);
    double end = MPI_Wtime();

    if (rank == 0) {
        printf("P0 received: ");
        for(int i=0; i<8; i++) printf("%d ", recv_arr[i]);
        printf("\nNonblocking time = %f s\n", end - start);
    }

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Prepare data. Rank 0 fills its vector with 10, 20, …, 80. Rank 1 fills its vector with 1, 2, …, 8.

Post operations. MPI_Isend starts a nonblocking send; MPI_Irecv starts a nonblocking receive. Both return request handles rather than waiting for the data transfer to finish.

Wait for completion. MPI_Waitall ensures both operations have completed before recv_arr is read or the buffers are reused.

Timing. MPI_Wtime measures the local duration around the exchange. A fair comparison requires repeated trials and a corresponding blocking implementation.

Check before submission: Only rank 0 prints the received vector and elapsed time; the code does not implement the requested blocking baseline, so it cannot yet compare the two approaches. Run with exactly two processes.

Lab analysis note: Nonblocking calls do not automatically make a program faster. They are most useful when independent computation can overlap communication, and both operations must complete before their buffers are accessed or reused.

Problem 8: Avoiding Deadlock

Question

Demonstrate the risk of deadlock with blocking communication and use an appropriate MPI communication routine to exchange messages safely.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int msg = (rank == 0) ? 100 : 200;
    int recv_msg;
    int target = (rank == 0) ? 1 : 0;

    // Corrected version to avoid deadlock: Sendrecv handles buffering
    MPI_Sendrecv(&msg, 1, MPI_INT, target, 0, &recv_msg, 1, MPI_INT, target, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

    printf("P%d sent %d, received %d\n", rank, msg, recv_msg);
    if (rank == 0) printf("Communication completed without deadlock.\n");

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Choose peer and message. Each of two ranks prepares a different integer and selects the other rank as target.

Exchange safely. MPI_Sendrecv combines a send and receive in one call, avoiding the classic pattern where both ranks block in MPI_Send before either posts a receive.

Report. Each rank prints the value it sent and the value it received.

Check before submission: This is the corrected MPI_Sendrecv version, not a demonstration of the unsafe version. Run with exactly two processes.

Problem 9: Distributed Sorting

Question

Divide an array among MPI processes, sort each local portion, exchange or combine the sorted portions, and produce a globally sorted array.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int cmp(const void *a, const void *b) { return (*(int*)a - *(int*)b); }

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int global_arr[8] = {45, 12, 78, 3, 56, 23, 89, 10};
    int local_arr[2];

    MPI_Scatter(global_arr, 2, MPI_INT, local_arr, 2, MPI_COMM_WORLD, 0);
    qsort(local_arr, 2, sizeof(int), cmp);
    printf("Local P%d: %d %d\n", rank, local_arr[0], local_arr[1]);

    // Simplified exact output mapping gathering to root to finish sorted output
    MPI_Gather(local_arr, 2, MPI_INT, global_arr, 2, MPI_INT, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        qsort(global_arr, 8, sizeof(int), cmp);
        printf("Sorted array: ");
        for (int i=0; i<8; i++) printf("%d ", global_arr[i]);
        printf("\n");
    }

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Input and comparator. The root holds eight integers. cmp returns a comparison result used by qsort to order integers in ascending order.

Scatter. MPI_Scatter gives each of four processes two integers.

Local sort. Each process sorts its two-element segment with qsort.

Gather and global sort. MPI_Gather collects the locally sorted segments at rank 0. The root then sorts all eight gathered values, guaranteeing a globally sorted output.

Check before submission: This solution performs local sorting, gathers everything, and sorts globally at root. It does not implement neighbor exchanges or a distributed odd-even merge sort. It is a simpler correct way to produce the final sorted array for exactly four processes and eight elements.

Problem 10: Producer–Consumer with MPI

Question

Use one producer process and two consumer processes. The producer sends eight values to consumers using point-to-point communication.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int items[8] = {10, 20, 30, 40, 50, 60, 70, 80};

    if (rank == 0) {
        for (int i = 0; i < 8; i++) {
            int target = (i % 2) + 1;
            MPI_Send(&items[i], 1, MPI_INT, target, 0, MPI_COMM_WORLD);
            printf("P0 produced: %d\n", items[i]);
        }
    } else {
        for (int i = 0; i < 4; i++) {
            int recv_item;
            MPI_Recv(&recv_item, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            printf("P%d consumed: %d\n", rank, recv_item);
        }
    }

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Producer data. Rank 0 owns the eight items to be sent.

Assign consumers. target = (i % 2) + 1 alternates destinations between rank 1 and rank 2.

Consume. Each consumer performs four receives from rank 0 and prints each received item.

Finish. The example is a fixed alternating schedule. It does not use MPI_ANY_SOURCE, despite the prose description in the supplied solution file.

Check before submission: The schedule assumes exactly three processes and four items per consumer. It is deterministic round-robin dispatch, not dynamic work assignment. Run with -np 3.

Problem 11: Broadcast Configuration Parameters

Question

The root initializes configuration values and broadcasts them to every process using MPI_Bcast.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int config[3];
    if (rank == 0) {
        config[0] = 1000; config[1] = 50; config[2] = 256;
    }

    MPI_Bcast(config, 3, MPI_INT, 0, MPI_COMM_WORLD);
    printf("P%d received: Iterations = %d, Threshold = %d, Block size = %d\n", rank, config[0], config[
1], config[2]);

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Configuration array. config stores three integers: iteration count, threshold, and block size.

Root initialization. Only rank 0 writes the configuration values.

Broadcast. MPI_Bcast sends the three integers from root 0 to every process in MPI_COMM_WORLD. All participating processes must call this collective.

Display. Each process prints the same configuration values.

Check before submission: The code works for any process count because the broadcast is collective. The configuration array is initialized only on root, as intended; MPI_Bcast distributes its values to all ranks.

Problem 12: Scatter, Compute, and Gather

Question

Distribute a vector using MPI_Scatter, square each local element, and collect the results at the root with MPI_Gather.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int vector[8] = {1, 2, 3, 4, 5, 6, 7, 8};
    int local[2];
    int result[8];

    MPI_Scatter(vector, 2, MPI_INT, local, 2, MPI_INT, 0, MPI_COMM_WORLD);
    local[0] *= local[0];
    local[1] *= local[1];
    printf("P%d: %d %d\n", rank, local[0], local[1]);

    MPI_Gather(local, 2, MPI_INT, result, 2, MPI_INT, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Gathered result: ");
        for (int i=0; i<8; i++) printf("%d ", result[i]);
        printf("\n");
    }

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Input and buffers. The root's vector has eight integers; each process receives two values into local.

Scatter. MPI_Scatter sends two consecutive integers to each process.

Local computation. Each local element is multiplied by itself to calculate its square.

Gather. MPI_Gather collects the two squared values from every rank in rank order at root 0.

Check before submission: The code assumes exactly four processes and two elements per process. MPI_Scatter's send count is two, so the vector length must equal 2 × number of processes.

Problem 13: Uneven Distribution with Scatterv and Gatherv

Question

Distribute a 5 × 3 matrix across three processes when the row count is not evenly divisible, multiply each element by two, and gather the result.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int matrix[15] = {1,2,3, 4,5,6, 7,8,9, 10,11,12, 13,14,15};
    int sendcounts[3] = {6, 6, 3};
    int displs[3] = {0, 6, 12};
    int local[6];

    MPI_Scatterv(matrix, sendcounts, displs, MPI_INT, local, sendcounts[rank], MPI_INT, 0,
MPI_COMM_WORLD);

    for (int i = 0; i < sendcounts[rank]; i++) local[i] *= 2;

    int final_mat[15];
    MPI_Gatherv(local, sendcounts[rank], MPI_INT, final_mat, sendcounts, displs, MPI_INT, 0,
MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Final matrix:\n");
        for (int i=0; i<15; i+=3) printf("%d %d %d\n", final_mat[i], final_mat[i+1], final_mat[i+2]);
    }

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Counts and displacements. sendcounts = {6, 6, 3} means ranks 0 and 1 receive six integers each and rank 2 receives three. displs = {0, 6, 12} gives each rank's starting offset in the flattened matrix.

Scatterv. MPI_Scatterv supports different receive counts for different ranks, unlike MPI_Scatter's equal-count distribution.

Local operation. Each process doubles the integers in its local segment.

Gatherv. MPI_Gatherv returns different amounts of data from each rank to the matching offsets in final_mat.

Check before submission: The arrays are hard-coded for three processes. Run with -np 3. MPI_Scatterv and MPI_Gatherv counts/displacements are measured in elements, not bytes. The provided flattened values represent five rows of three columns.

Problem 14: Global Sum, Maximum, Minimum, and Average

Question

Each process computes local statistics for its values. Use MPI_Reduce to calculate global sum, maximum, and minimum, then compute the average.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int local_arr[3] = {rank*30 + 10, rank*30 + 20, rank*30 + 30};
    int local_sum = local_arr[0] + local_arr[1] + local_arr[2];
    int local_max = local_arr[2];
    int local_min = local_arr[0];

    int g_sum, g_max, g_min;
    MPI_Reduce(&local_sum, &g_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Reduce(&local_max, &g_max, 1, MPI_INT, MPI_MAX, 0, MPI_COMM_WORLD);
    MPI_Reduce(&local_min, &g_min, 1, MPI_INT, MPI_MIN, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Global Sum = %d\nGlobal Maximum = %d\nGlobal Minimum = %d\nAverage = %.2f\n",
               g_sum, g_max, g_min, (float)g_sum/12.0);
    }

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Local values. Each rank generates three integers. It calculates its local sum and, because values are increasing within each segment, its local maximum and minimum.

Three reductions. MPI_Reduce with MPI_SUM, MPI_MAX, and MPI_MIN combines one local statistic from every process at root 0.

Average. The root divides the global sum by the total number of values. With four ranks and three values per rank, the denominator is 12.

Check before submission: The code assumes four processes and three values per process because the average divides by 12. For other sizes, calculate the count dynamically and reduce it too. Local min/max shortcuts are valid only because the generated local values are ordered.

Problem 15: Trapezoidal Rule Integration

Question

Approximate the integral of f(x) = x² over [0, 1] using the trapezoidal rule. Each process computes a subinterval and MPI_Reduce combines partial integrals.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

double f(double x) { return x * x; }

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int n = 1000;
    double a = 0.0, b = 1.0;
    double h = (b - a) / n;
    int local_n = n / size;

    double local_a = a + rank * local_n * h;
    double local_b = local_a + local_n * h;

    double local_int = (f(local_a) + f(local_b)) / 2.0;
    for (int i = 1; i < local_n; i++) {
        local_int += f(local_a + i * h);
    }
    local_int *= h;

    double total_int;
    MPI_Reduce(&local_int, &total_int, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) printf("Numerical integral: %f\n", total_int);

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Function and interval. f(x) returns x². The integral is approximated on [a,b] with n subintervals of width h = (b–a)/n.

Partition. Each rank gets local_n = n/size intervals and computes its local start and end points.

Trapezoidal rule. The endpoints contribute half weight; interior points contribute full weight. Multiplication by h scales the sum into an area estimate.

Reduction. MPI_Reduce adds each process's partial integral at rank 0.

Check before submission: The code assumes n is divisible by the number of processes; otherwise n/size drops leftover intervals. With n=1000 and four processes this is fine. A general solution should distribute remainder intervals and report the exact error.

Lab analysis note: For f(x)=x² on [0,1], the exact integral is 1/3. The numerical result should approach 0.333333 as the interval count increases.

Problem 16: Prefix Sum with MPI_Scan

Question

Each process stores a local value. Use MPI_Scan with MPI_SUM to compute the inclusive prefix sum across processes.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int local_val = (rank + 1) * 10;
    int prefix_sum;

    MPI_Scan(&local_val, &prefix_sum, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
    printf("P%d: Local value = %d, Prefix sum = %d\n", rank, local_val, prefix_sum);

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Local values. Rank r uses (r + 1) × 10, so the values are 10, 20, 30, 40 for four ranks.

Inclusive prefix. MPI_Scan with MPI_SUM returns the sum of local values from rank 0 through the calling rank.

Interpretation. For four ranks, the results are 10, 30, 60, 100. Unlike MPI_Reduce, MPI_Scan returns a prefix result to every process.

Check before submission: MPI_Scan computes the prefix result, but the supplied solution does not include the requested manual point-to-point comparison.

Problem 17: All-to-All Communication

Question

Each process sends one integer to every process, including itself, using MPI_Alltoall. Observe how the received values depend on the source process.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int send_data[4] = {rank*10, rank*10 + 1, rank*10 + 2, rank*10 + 3};
    int recv_data[4];

    MPI_Alltoall(send_data, 1, MPI_INT, recv_data, 1, MPI_INT, MPI_COMM_WORLD);

    printf("P%d received: %d %d %d %d\n", rank, recv_data[0], recv_data[1], recv_data[2], recv_data[3]);

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Send data. Each process builds four values based on its rank. Element j is the value intended for destination rank j.

All-to-all. MPI_Alltoall sends one integer to every destination and receives one integer from every source. The receive buffer is ordered by source rank.

Output. For example, rank 0 receives the first element from each source: 0, 10, 20, 30.

Check before submission: The code uses four send and receive elements per process and therefore assumes four processes. It does not time communication; the assignment asks for scaling tests with different process counts and message sizes.

Lab analysis note: To study scaling, repeat with process counts 2, 4, 8, and 16 and vary the message count. The current fixed-size arrays must be adapted before changing the process count.

Problem 18: Global Histogram with MPI_Allreduce

Question

Each process builds a local histogram for values 0–9. MPI_Allreduce combines all local histograms into a global frequency table.

Solution Code (C / MPI)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int data[4][4] = {{1,2,2,3}, {2,3,4,4}, {1,4,5,5}, {0,2,5,9}};
    int local_hist[10] = {0};
    int global_hist[10];

    for (int i = 0; i < 4; i++) {
        local_hist[data[rank][i]]++;
    }

    MPI_Allreduce(local_hist, global_hist, 10, MPI_INT, MPI_SUM, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Value Frequency\n");
        for (int i = 0; i < 10; i++) {
            printf("%d %d\n", i, global_hist[i]);
        }
    }

    MPI_Finalize();
    return 0;
}
```

Code Explanation

Local data and histogram. Each rank processes one row of the data array. local_hist starts with ten zero counters, one for each possible value 0–9.

Count occurrences. For every value v in the local row, local_hist[v] is incremented.

Combine histograms. MPI_Allreduce with MPI_SUM adds each bin across all processes and makes the global histogram available to every rank.

Display. Rank 0 prints the frequency for every value from 0 through 9.

Check before submission: The input is arranged as four rows of four values, one row per rank, so run with -np 4. The sample output in the lab sheet appears inconsistent with the listed data; the frequencies produced by the code are the ones implied by the data array.